



REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

UNIVERSITY OF BUEA

COLLEGE OF TECHNOLOGY

DEPARTMENT OF COMPUTER
ENGINEERING



REPUBLIQUE DU CAMEROUN

PAIX-TRAVAIL-PATRIE

UNIVERSITE DE BUEA

COLLEGE DE TECHNOLOGIE

DEPARTEMENT DE GENIE
INFORMATIQUE

**JOBLINK: AN AI-POWERED JOB AGGREGATOR AND RESUME
BUILDER**

BY

NSAICHIE BERINYUY NYEANCHI

CT23A132

Joblink is a web-based platform design to bridge the gap between
Cameroon tech professionals and remote job opportunities offered by
western countries.

COURS INSTRUCTOR: MR. KOMETA DENNIS

JUNE 2026

2025/2026 ACADEMIC YEAR

S

COURSE: CEC418 – Software Construction and Evolution

STUDENT NAME: NSAICHIE BERINYUY NYEANCHI

MATRICULE NUMBER: CT23A132

PROJECT REPORT: Joblink – AI-POWERED JOB AGGREGATOR AND
RESUME BUILDER

Version: Version 1 (Cameroon Testing Ground)

Table of Contents

1. PROJECT OVERVIEW	2
2. DEVELOPMENT STRATEGY	2
2.1 Architecture Decisions	2
2.2 Core Product Mechanics	3
2.3 Monetization Strategy (V1)	3
3. VERIFICATION AND VALIDATION (V&V).....	3
3.1 Verification.....	4
3.2 Validation	4
4. DEVOPS STRATEGY AND TOOLING.....	5
4.1 Containerization – Docker	5
4.2 Configuration Management – Ansible	6
4.3 CI/CD Pipeline – GitHub Actions.....	6
4.4 Cloud Infrastructure – AWS Free Tier	7
5. MAINTENANCE STRATEGY	8
6. DEVOPS TOOLS SUMMARY	9
7. CONCLUSION	10
REFENCES	10

1. PROJECT OVERVIEW

Joblink is a web-based platform designed to bridge the gap between Cameroonian tech professionals and remote job opportunities offered by Western companies. The platform operates as a hybrid AI-powered job aggregator and resume builder, targeting bilingual (English/French) job seekers in Cameroon.

The system scrapes remote job listings from Western tech startups and media organizations, presents them to job seekers, and uses AI to instantly generate tailored resumes and cover letters on demand. Version 1 (V1) focuses exclusively on the Cameroonian market as a testing and validation ground before scaling Pan-African in Version 2.

Technology Stack:

- Frontend: React (apps/web)
- Backend: Node.js with TypeScript (apps/api)
- Database: Supabase (PostgreSQL)
- Monorepo: pnpm workspaces

2. DEVELOPMENT STRATEGY

The development of Joblink followed an incremental and iterative approach, structured around a V1-to-V2 product lifecycle matrix. V1 was scoped deliberately to reduce risk, stabilize AI API costs, and validate core mechanics within a known local market.

2.1 Architecture Decisions

The project was structured as a monorepo using pnpm workspaces, containing two primary applications:

- apps/web: The React frontend, responsible for job discovery, user profiles, and AI-generated document display.
- apps/api: The TypeScript Node.js backend, organized into lib, middleware, routes, and services layers.

This separation enforces clear boundaries between presentation logic and business logic while allowing shared utilities across packages.

2.2 Core Product Mechanics

The Aggregator Data Pipeline: Background scrapers continuously target Western tech job boards and 'Worldwide Remote' listings, caching job metadata locally to reduce redundant AI processing costs.

The Hybrid Bridge UX: When a user selects a job and clicks 'Apply with AI', the system reads the scraped job description alongside the user's stored profile and invokes the AI model to produce a tailored resume (PDF) and cover letter. The user downloads the resume, copies the cover letter, and is redirected to the original application page for manual submission.

Localization Features:

- **Bilingual Resume Polishing:** AI translates French academic and professional phrasing into standard US-equivalent terminology.
- **Remote Ready Badge:** Users earn a verified infrastructure badge by uploading proof of backup power and completing an in-app internet speed test.

2.3 Monetization Strategy (V1)

B2C – Pay-As-You-Go Token System: New users receive 2 free AI credits. Additional credits are purchased via Mobile Money (Orange Money / MTN MoMo). One credit equals one AI-generated CV and cover letter.

B2B – Paid Job Boosts: Job postings remain free initially to populate the platform. Employers pay a flat fee of \$49 per post to receive priority placement and targeted WhatsApp/Telegram push notifications to matched Remote Ready candidates.

3. VERIFICATION AND VALIDATION (V&V)

Verification and Validation were managed across multiple layers of the software development lifecycle to ensure correctness, reliability, and fitness for purpose.

3.1 Verification

Verification activities ensured that the system was being built correctly according to its design specification.

Static Code Analysis: ESLint with TypeScript rules was applied across the apps/api and apps/web codebases to enforce consistent code style, catch type mismatches, and identify unreachable code paths before runtime.

TypeScript Compilation: The TypeScript compiler (tsc) served as a continuous verification tool, flagging type errors and interface mismatches during development. All service layer functions, route handlers, and middleware were typed explicitly.

Code Review: A structured peer review process was followed before merging any feature branch into main. Reviews checked route logic, middleware ordering, Supabase query correctness, and AI prompt structure.

3.2 Validation

Validation activities confirmed that the software met real user needs and business requirements.

Manual End-to-End Testing: Complete user flows were tested manually, including job discovery, AI document generation, credit deduction, Mobile Money integration, and the Remote Ready badge workflow.

API Endpoint Testing: Postman was used to validate all REST endpoints, verifying correct HTTP status codes, response payloads, authentication headers, and error handling for invalid inputs.

AI Output Validation: Generated resumes and cover letters were reviewed by bilingual test users to assess translation accuracy, professional tone, and relevance to the target job description.

Database Integrity Checks: Supabase row-level security policies and foreign key constraints were validated to ensure unauthorized data access was blocked and relational integrity was preserved across user profiles, job listings, and credit transactions.

4. DEVOPS STRATEGY AND TOOLING

The DevOps strategy for Joblink was built around four pillars: containerization, configuration management, continuous integration and delivery, and cloud infrastructure management. All tools were selected to support reproducible, automated, and auditable deployments.

4.1 Containerization – Docker

Docker was used to containerize the Node.js backend, packaging the application and all its dependencies into a single portable image. This eliminated environment inconsistencies between local development and production.

A multi-stage Dockerfile was written for apps/api, using a Node 18 Alpine base image to minimize image size. The pnpm workspace structure required copying workspace manifests and the api package separately before running a frozen install, ensuring reproducible dependency resolution.

Docker was chosen because:

- It guarantees identical runtime environments across all stages.
- Images can be versioned by Git commit SHA, enabling precise rollback.
- It integrates natively with AWS ECR and EC2.

Result: The Docker image for Joblink -backend built successfully and ran locally on port 3000, returning expected API responses. The image was tagged and pushed to AWS ECR.

4.2 Configuration Management – Ansible

Ansible was used to automate the configuration of the AWS EC2 production server. Rather than manually SSHing into the server to install Docker, pull images, and start containers, an Ansible playbook codified all these steps as repeatable, version-controlled tasks.

The playbook performed the following operations on the EC2 instance:

- Updated apt package lists
- Installed Docker and the AWS CLI
- Authenticated with AWS ECR
- Pulled the latest Joblink -backend image
- Stopped and removed any previous container instance
- Started a new container with environment variables injected at runtime

Ansible was chosen because:

- It is agentless, requiring only SSH access to the target server.
- Playbooks are human-readable YAML, making them easy to audit and maintain.
- It enforces idempotency, meaning repeated runs produce the same result without side effects.

Result: The Ansible PLAY RECAP confirmed all tasks completed with status ok or changed, with zero failures. The EC2 server was fully provisioned and the backend container was running after a single playbook execution.

4.3 CI/CD Pipeline – GitHub Actions

A GitHub Actions workflow was implemented to automate the full build and deployment process on every push to the main branch. The pipeline was split into two parallel jobs: backend deployment and frontend deployment.

Backend Job:

1. Checks out source code
2. Authenticates with AWS using IAM credentials stored as GitHub Secrets

3. Logs into Amazon ECR
4. Builds the Docker image and tags it with the Git commit SHA
5. Pushes the image to ECR (tagged as both commit SHA and latest)
6. SSHes into the EC2 instance and performs a zero-downtime container replacement

Frontend Job:

1. Sets up pnpm and Node.js 18
2. Installs all workspace dependencies
3. Builds the React app with the production API URL injected as an environment variable
4. Syncs the build output to the AWS S3 bucket
5. Invalidates the CloudFront CDN cache to serve fresh content immediately

GitHub Actions was chosen because:

- It integrates natively with the GitHub repository, requiring no external CI server.
- Secrets management is built-in, keeping credentials out of the codebase.
- The free tier provides 2,000 minutes per month, sufficient for a V1 project.

Result: The CI/CD pipeline ran successfully on the first clean push to main. Both the backend and frontend jobs completed without errors. The frontend became accessible via the CloudFront distribution URL and the backend API was reachable on the EC2 public IP within minutes of the push.

4.4 Cloud Infrastructure – AWS Free Tier

The production infrastructure was deployed entirely on AWS using free tier eligible services:

- EC2 t2.micro (Ubuntu 24.04 LTS): Hosts the Docker container running the Node.js backend. Free for 12 months.
- Amazon ECR: Stores versioned Docker images. 500MB free storage per month.

- Amazon S3: Hosts the compiled React frontend as a static website. 5GB free storage.
- Amazon CloudFront: Serves the frontend globally with HTTPS. 1TB data transfer free per month.
- Supabase Cloud: Manages the PostgreSQL database. Free tier supports the V1 user load.

IAM: A dedicated Joblink -deployer IAM user was created with scoped permissions, following the principle of least privilege. Root account access is protected with MFA.

5. MAINTENANCE STRATEGY

Maintenance of the Joblink platform is managed through a combination of automated processes and infrastructure design decisions.

Automated Redeployment: Any push to the main branch triggers the GitHub Actions pipeline, ensuring the production environment always reflects the latest stable code without manual intervention.

Container Self-Healing: The Docker container on EC2 is started with the `--restart unless-stopped` flag, meaning it automatically restarts after a crash or server reboot without operator action.

Rollback Capability: Because Docker images are tagged by Git commit SHA and stored in ECR, any previous build can be redeployed by pulling a specific image tag. This provides a reliable rollback mechanism in case a new deployment introduces a regression.

Database Maintenance: Supabase Cloud manages automated backups, connection pooling, and PostgreSQL version updates on the free tier, removing database maintenance overhead from the development team.

Frontend Cache Management: CloudFront cache invalidation is executed automatically at the end of every frontend deployment job, ensuring users always receive the latest build without stale cache issues.

Monitoring: EC2 instance health is monitored through the AWS Console. Future iterations will integrate AWS CloudWatch alarms for CPU and memory thresholds.

6. DEVOPS TOOLS SUMMARY

Tool	Purpose	Why Used	Result
Docker	Containerize Node.js backend	Environment consistency, versioned images, AWS ECR compatibility	Image built and pushed to ECR successfully
AWS ECR	Store Docker images	Native AWS integration, image versioning by commit SHA	Joblink -backend repository created and populated
AWS EC2 (t2.micro)	Host backend container	Free tier, full server control, SSH + Ansible compatible	Backend API live on public IP
Ansible	Configure EC2 server	Agentless, idempotent, infrastructure as code	PLAY RECAP: all tasks ok/changed, zero failures
GitHub Actions	CI/CD pipeline	Native GitHub integration, free minutes, secrets management	Full pipeline passed: Docker build, ECR push,

			EC2 deploy, S3 sync
--	--	--	---------------------

7. CONCLUSION

Joblink V1 successfully demonstrates a complete software construction and evolution lifecycle applied to a real-world product targeting the Cameroonian job market. The platform integrates AI-powered document generation, bilingual localization, mobile payment, and infrastructure verification into a cohesive user experience.

From a DevOps perspective, the project achieves full automation from code commit to live deployment using Docker, Ansible, GitHub Actions, and AWS free tier services. The V1 architecture is deliberately scoped to validate assumptions cheaply and build a stable foundation before the Pan-African V2 expansion.

The combination of containerization, configuration management, and CI/CD pipelines ensures that Joblink can be maintained, scaled, and evolved efficiently as the platform grows.

REFERENCES